PRJ-18: Real-Time University Bus Tracking System

WEEK 7

PROJECT PROGRESS DOCUMENTATION

Student Name	**Birendra Prasad (25PG00018)**
Programme	Master of Computer Applications (MCA)
Specialization	Cyber Security
Repository	github.com/BirendraPrasad/University-Bus-Tracking

1. Introduction

The University Bus Tracking System is a web-based application intended to support university transportation administration. During Week 7, work focused on connecting the existing Admin Dashboard to the FastAPI backend and PostgreSQL database, and implementing the Add Driver workflow.

2. Objectives

- Connect the Admin Dashboard to existing API endpoints.
- Display user, bus, driver, and route records retrieved from the backend.
- Implement driver creation through an authenticated API request.
- Verify that new driver information is stored and displayed.
- Test the workflow and correct frontend integration errors.

3. Technology Stack

Technology	Purpose
HTML5 / CSS3	Dashboard structure and presentation
JavaScript	Frontend interactions and API integration
Python / FastAPI	Backend application and REST APIs
PostgreSQL	Relational database
Uvicorn	Run the FastAPI application
FastAPI Swagger UI	API testing and verification
Git / GitHub	Source code management

4. System Architecture

The application uses a client–server architecture. The administrator interacts with the dashboard; JavaScript sends authenticated HTTP requests to FastAPI; the backend validates permissions and reads or writes records in PostgreSQL; the API response is then used to update the dashboard.

Administrator → Admin Dashboard (HTML, CSS, JavaScript) → FastAPI REST API → PostgreSQL

5. Admin Dashboard API Integration

The dashboard was connected to existing GET endpoints so that records are loaded from the backend rather than relying only on static content.

Method	Endpoint	Purpose
GET	/users/	Retrieve user records
GET	/users/buses	Retrieve bus records
GET	/users/drivers	Retrieve driver records
GET	/users/routes	Retrieve route records
GET	/users/routes/{id}/stops	Retrieve stops for a selected route

Integrated behaviour includes data loading, search, applicable filters, pagination, refresh controls, loading indicators, error messages, overview counters, and viewing stops for a selected route. This does not imply every module supports create, edit, and delete operations.

6. Add Driver Functionality

The Add Driver form collects name, email, password, licence number, and phone. It sends a JSON POST request to POST /users/drivers with the administrator's bearer token. The backend validates administrator access, creates the user and driver profile, and stores records in PostgreSQL. Password hashing is handled by the backend.

After a successful response, the frontend displays a success notification, resets the form, closes the dialog, and reloads dashboard data. If the request fails, an error notification is displayed.

- api(path) retrieves data using GET requests.
- apiPost(path, payload) submits JSON data using POST requests.
- toast(message, kind) displays success/error notifications.
- load(name) and loadAll() load and refresh dashboard data.
- The driverForm submit handler validates and submits the driver form.

7. Database Integration and Verification

The PostgreSQL database is university_bus_tracking. The drivers table contains driver_id, user_id, license_number, and phone. Name and email are stored in the related users table, so a JOIN is required to retrieve complete driver details.

```
SELECT d.driver_id, d.user_id, u.name, u.email, d.license_number, d.phone
FROM drivers d
JOIN users u ON d.user_id = u.user_id
ORDER BY d.driver_id;
```

The initial query showed Rakesh Kumar (driver_id 1; user_id 5; driver@university.com; licence DL-JH-12345; phone 9876543210). After the Add Driver form was corrected and tested, the dashboard displayed two driver records: Rakesh Kumar and Abhay (abhay123@gmail.com). The screenshot showed "No bus" for the new driver, indicating that a bus had not yet been assigned.

8. Testing and Validation

Test	Expected behaviour	Observed status
GET /users/drivers	Return driver records	HTTP 200 observed in Swagger UI
Load Drivers table	Display existing drivers	Verified
Submit Add Driver form	Create a driver account	Verified after frontend fix
Database verification	Confirm stored driver details	SQL JOIN query executed
Refresh dashboard	Show latest driver records	New driver displayed
Bus assignment display	Show bus or "No bus"	"No bus" shown for new driver

During debugging, a JavaScript ReferenceError occurred because the driver form handler had been placed outside the dashboard's main JavaScript function. Moving the handler inside the function corrected the integration issue.

9. Results and Achievements

- Connected the Admin Dashboard to existing FastAPI endpoints.
- Loaded user, bus, driver, and route data from the backend.
- Implemented and tested the Add Driver form submission.
- Displayed the newly created driver in the Drivers table after refreshing data.

- Verified driver records through PostgreSQL and the API testing interface.
- Added success/error notifications.

10. Current Status and Scope

Module	Week 7 status
Admin Dashboard API data integration	Working for existing GET endpoints
User listing	Connected to existing GET endpoint
Bus listing	Connected to existing GET endpoint
Driver listing	Connected to existing GET endpoint
Add Driver	Implemented and tested
Routes listing / view stops	Existing GET integration available
Add Student	Pending
Bus creation/editing	Pending verification and implementation
Route creation/editing	Pending verification and implementation
Trip management	Pending
Live GPS tracking	Not verified as completed
Kafka integration	Not verified as completed

The dashboard's JavaScript file identifies this stage as a read-only integration with existing API data. Accordingly, this report does not claim complete CRUD functionality for all modules.

11. Challenges and Solutions

JavaScript reference error: The browser reported "ReferenceError: $ is not defined." The driver form handler was moved inside the main dashboard JavaScript function, where helper and API functions are in scope.

Driver not appearing: The API response, frontend form submission, and database records were checked. The handler placement was corrected and the dashboard refresh workflow was tested.

Database column mismatch: The drivers table did not contain name or email columns. The table schema was inspected using \d drivers, and a JOIN with the users table was used.

Swagger authentication: The protected API returned "Not authenticated" when accessed without credentials. The endpoint was tested after configuring administrator authentication in Swagger UI.

12. Pending Work and Next Steps

- Implement and test Add Student functionality.
- Implement and verify bus creation and editing workflows.
- Implement and verify route creation and editing.
- Complete trip management operations.
- Develop and verify live GPS tracking.
- Verify any required real-time/Kafka integration against project requirements.
- Perform end-to-end tests for the remaining modules.
- Update documentation as additional features are completed.

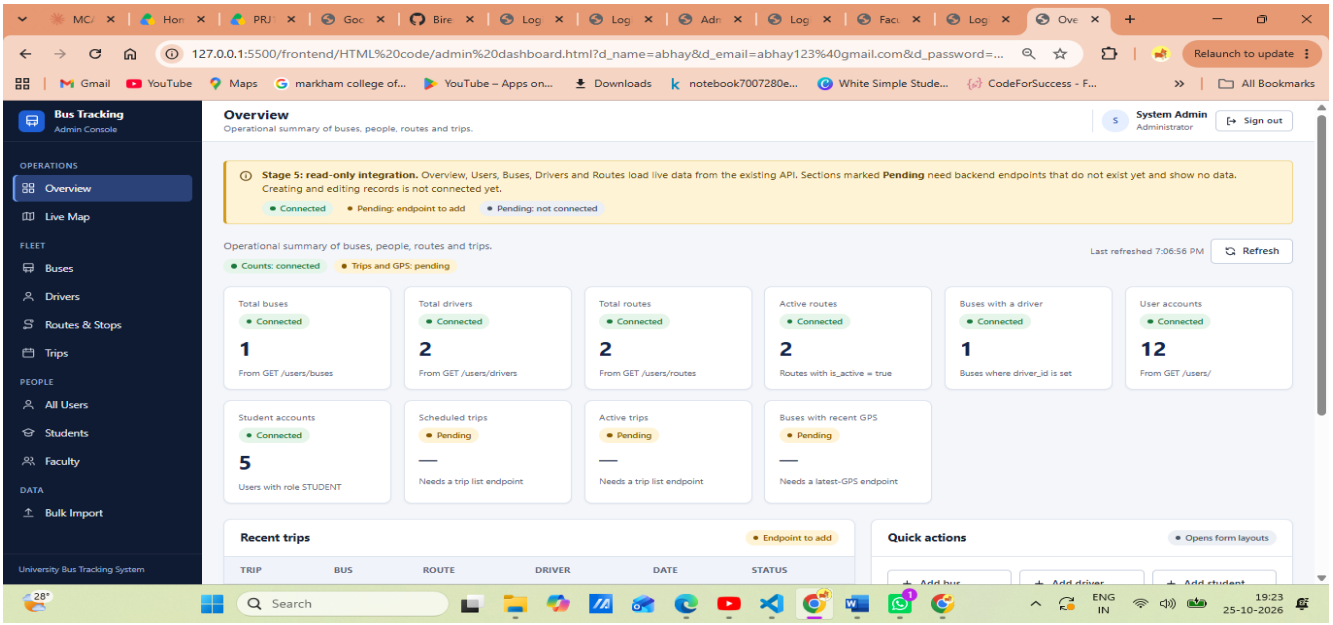
13. Conclusion

Week 7 work focused on integrating the Admin Dashboard with the existing FastAPI backend and PostgreSQL database. The Add Driver workflow was implemented and tested: a driver can be submitted through the dashboard, saved through the API, and displayed in the Drivers table after data refresh. This provides a foundation for continuing work on the remaining administrative modules and tracking functionality.

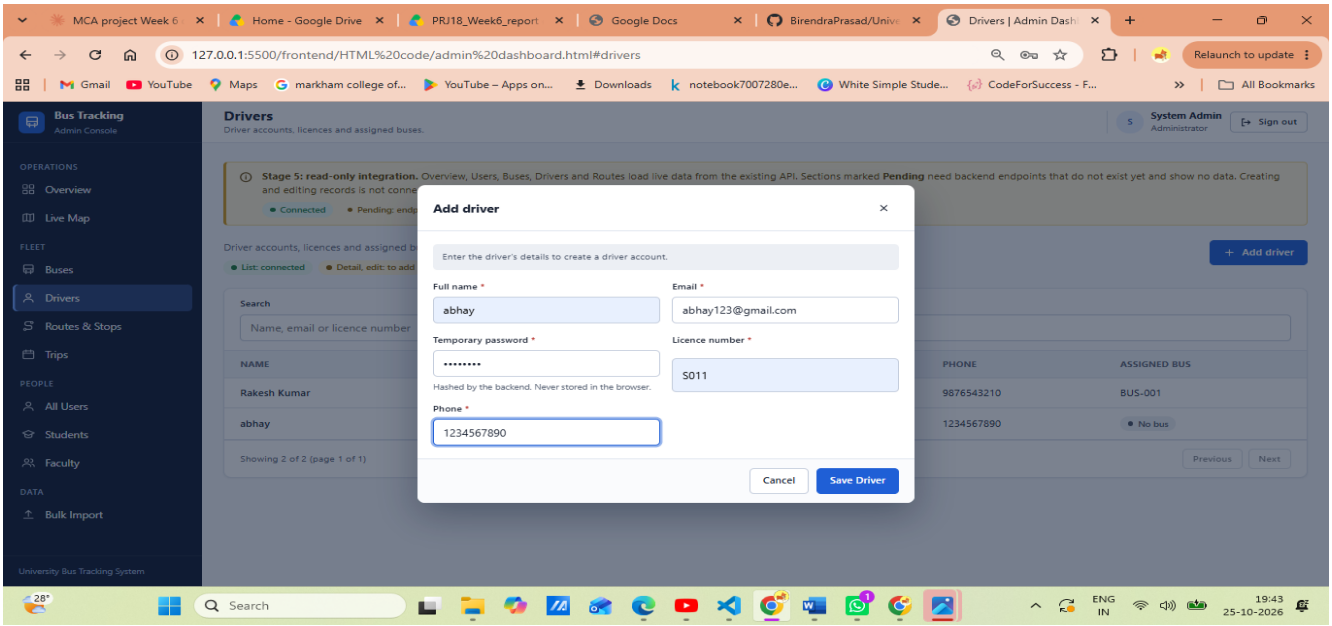
Appendix A. Screenshot Evidence

Insert the screenshots captured during implementation under the matching captions. The current report file does not embed the original screenshot assets automatically; add the originals before submission.

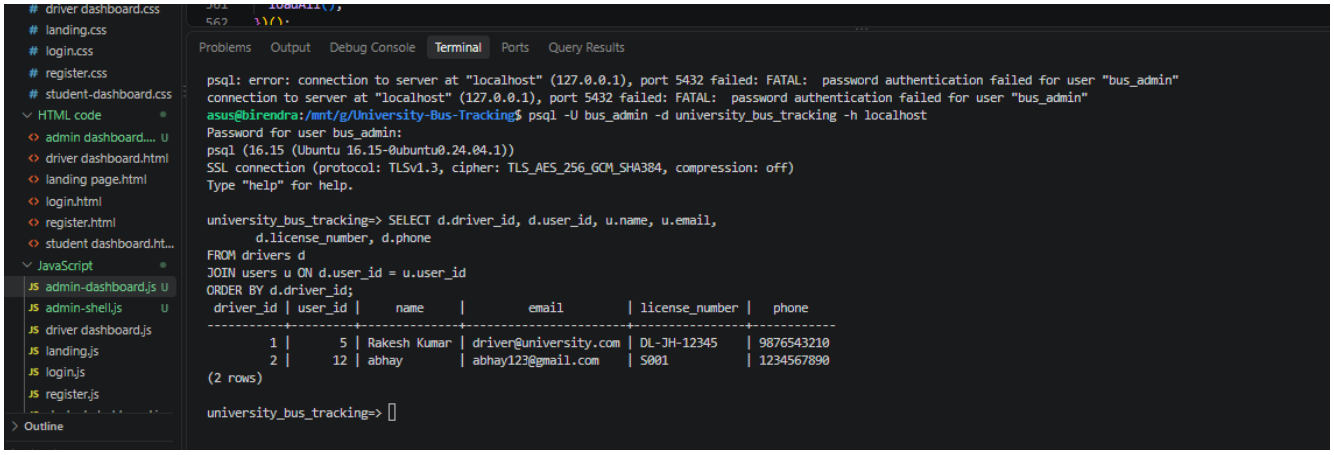
- Figure A1. Admin Dashboard — Drivers module showing the driver records.



- Figure A2. FastAPI Swagger UI — successful GET /users/drivers response (HTTP 200).



- Figure A3. PostgreSQL — SQL JOIN query and driver records.



- Figure A4. Final dashboard — Rakesh Kumar and Abhay visible in the Drivers table.

